

Hash Functions

Recall: In our last few classes, we've learned that we can construct a MAC for any single block message by using PRFs, and larger messages can be encrypted with a MAC by combining block ciphers with the modes of permutations. AES is an example of a block cipher that's a PRP, and we can utilize AES to construct CBC-MAC. In the construction of the secure CBC-MAC, we must also pad the message by concatenating the length of the message at the very beginning of the message. Though CBC-MAC is secure in this instance, it cannot be used until the length of the message is known.

Thus, we consider how we can construct a MAC using Hash. We learned that, if we have a Collision-Resistant Hash Function (CRHF), we can create a one block message from an arbitrarily long message by Hashing. Then, we can MAC the resulting message in order to create a MAC that will be secure for the original message. This MAC of an arbitrarily long function can be represented as Π_{BMac} .

For more information, refer to the MAC and CRHF notes both located on the canvas page under files, then notes, labeled MAC.pdf and CRHF.pdf.

1 MAC and MAC Forge

1.1 Message Authentication Code

Definition 1.1 (Message Authentication Code (MAC)). Consider the example where Alice is attempting to send a secret message to Bob. Bob needs to be confident in both the integrity and authenticity of the message. In other words, Bob must be able to ensure that the message hasn't been tampered with (integrity) and that the message is truly sent by Alice (authenticity). The scheme that guarantees integrity and authenticity is what we refer to as the MAC. This scheme is what we refer to as the MAC.

Syntactical Representation of MAC Algorithm

Setup $\rightarrow K_{mac}$

Sign $(K_{mac}, m) \rightarrow \sigma$

Verify $(K_{mac}, m, \sigma) \rightarrow 0 \text{ or } 1$

1.2 MAC Forge Game

A PPT Adversary A can break MAC security if they're able to win the MAC Forge Game, which is represented below:

Setup Phase: The challenger generates a secret key K using the algorithm: $K \leftarrow \Pi_{mac} * \text{Gen}(1^n)$.

Query Phase: The adversary A makes queries to two oracles. The sign oracle takes a queried m and returns the valid $\sigma = \text{Sign}(K, m)$. The verify oracle takes queried (m, σ) pairs and uses $\text{Verify}(K, m, \sigma)$ to check if σ is a valid MAC for m under the key K . If it is, the verify oracle outputs 1. If it isn't, it outputs 0.

Guess Phase: The PPT Adversary A sends a forged message (m^*, σ^*) to the challenger. Consider Q to be the query space. If $m^* \notin Q$, meaning that m^* was not previously queried, and $\text{Verify}(K, m^*, \sigma^*) = 1$, then the challenger outputs 1. Else, the challenger outputs 0.

If $P[\text{Mac Forge}(A, \Pi_{mac}) \rightarrow 1] = \text{neg}(n)$, then Π_{mac} is secure.

2 Security of Π_{BMac}

2.1 Proving Security

Proof.

Claim: Π_{BMac} is secure.

$\Rightarrow$ A MAC is secure if $\exists$ no PPT adversary which can produce a valid forgery m^*, σ^* on Π_{BMac} , where $m^* \notin Q$, and Q is the query space. If there exists a PPT adversary A that is able to produce a valid forgery m^*, σ^* on Π_{BMac} , where $m^* \notin Q$. Then, $\exists$ two options:

1. Adversary A has found a collision on the hash function of the message, H^s such that $m^* \notin Q$, but there exists some other $m_i \in Q$ such that $H^s(m^*) = H^s(m_i)$. In other words, there exists two different messages in the message space that yield the same hash function.
2. Adversary A has not found a collision on H^s , but has somehow found a way to compute $\text{Mac}_k(H^s H m^*)$ such that $H^s(m^*) \notin H^s(Q) := \{H^s(m) | m \in Q\}$. If A has figured out a way to compute the MAC on a new, unqueried m^* , then A can easily create a forgery m^*, σ^* on Π_{BMac} .

If we are in the first scenario, then adversary A is able to break the collision resistance of the hash function. If we are in the second scenario, then adversary A is able to break MAC security. Thus, any PPT adversary A that's able to produce a valid forgery on Π_{BMac} falls into one of these two types.

From the MAC Forge game: $P[\text{Mac Forge}(A, \Pi_{BMac}) = 1] = P[A \text{ produces a valid forgery } (m^*, \sigma^*) \text{ s.t. } m^* \notin Q] = P[A \text{ finds forgery and finds collision}] + P[A \text{ finds forgery and doesn't find collision}]$

In order to compute the total advantage of winning this game and breaking the security of Π_{BMac} , we must sum the advantages of the adversary in the first scenario and the second scenario. If both the hash and the mac are secure, then Adv_1 (the advantage in the first scenario, when the collision is found) and Adv_2 (the advantage in the second scenario, when the collision is not found but the forgery is still produced) should both be negligible, yielding a total advantage that is also negligible. If the sum of Adv_1 and Adv_2 is negligible, then Π_{BMac} is secure. Now, let us prove the negligibility of both of these advantages.

Advantage 1 Proof.

Claim: If the hash function Π_H is secure, then Adv_1 is negligible.

$\Rightarrow$ Let us prove this by contrapositive; if Adv_1 is non-negligible, then we can find a collision on Π_H . We can prove this by constructing an adversary C_A that internally simulates adversary A in order to break hash security. Then, the game is as follows:

Setup Phase: The challenger initializes the hash function by sampling s from $H.\text{Setup}(1^n)$. Adversary C_A is then given s , and C_A samples the small MAC key K

Query Phase: Adversary A queries messages m , and C_A hashes the messages using the hash function H to produce the signature σ , and sends it to A . Then, A sends $C_A(m, \sigma)$, and C_A uses the verification function to respond to the query. These query phases can be repeated polynomially many times.

Challenge Phase: A sends a forgery (m^*, σ^*) to C_A , where $m^* \notin Q$.

Guess Phase: C_A checks the forgery with the verification function. If $m^* \notin Q$ and the forgery is successful, then the reduction is able to find a collision, meaning that there exists another message $m_i \in Q$ such that $H^s(m^*) = H^s(m_i)$. Then C_A sends the guess (m^*, m_i) to the challenger.

We thus see that the probability that C_A finds the collision is equivalent to the probability that A constructs the valid forgery, so $P[\text{Hash-Coll}(C, \Pi_H) \rightarrow 1] = \text{Advantage 1}$. We've assumed that adversary A is able to find the forgery with non-negligible advantage, thus, adversary C_A is also able to find a collision with non-negligible advantage. Then, the contrapositive is proven. Thus, $\text{Adv}_1 \leq \text{neg}(n)$. $\square$

Advantage 2 Proof.

Claim: If the MAC function is secure, then Adv_2 is negligible.

$\Rightarrow$ In the second scenario, the adversary A on Π_{BMac} has somehow produced a σ^* on a message that it

has not queried, and A has not found a collision. Thus, we see that A has produced a valid forgery on this 1-block MAC. We prove this to be true by constructing a PPT adversary D_A on Π_{Mac} that is able to produce valid forgeries and break MAC security. We represent this with the following game:

Setup Phase: The challenger (on Π_{Mac}) initializes the game by sampling a MAC key K through $\Pi_{Mac} * \text{Gen}(1^n)$, and the attacker D_A samples hash keys s .

Query Phase: Attacker A queries messages m and attacker D_A hashes the queries using $H^s(M)$ and forwards the hashes to the challenger. The challenger then sends D_A the signature σ , which is then forwarded to the attacker A . The attacker A then queries m, σ pairs, and D_A hashes them and sends them to the challenger. The challenger uses the verification function on them and outputs 0 or 1 to D_A , which D_A forwards to A . Both of these query phases (sign and verify) can be repeated polynomially many times by attacker A .

Challenge Phase: Attacker A sends a forgery (m^*, σ^*) to D_A .

Guess Phase: D_A hashes m^* to obtain $H^s(m^*)$ and sends $(H^s(m^*), \sigma^*)$ to the challenger. If the forgery is valid, then D_A wins the game.

We can write that $P[\text{Mac Forge}(D, \Pi_{Mac}) \rightarrow 1] = P[\text{Mac Forge}(A, \Pi_{Bmac}) \rightarrow 1 \text{ and } A \text{ did not find a collision}]$. The right hand side of the probability represents the advantage of the attacker in the second scenario. The left hand side of the probability is negligible if Π_{mac} is secure. Thus, we see that if Π_{mac} is secure, advantage 2 is negligible. Thus, $\text{Adv}_2 \leq \text{neg}(n)$. $\square$

Finally, we see that $P[\text{Mac Forge}(A, \Pi_{BMac})] = \text{Adv}_1 + \text{Adv}_2 \leq \text{neg}(n)$. This proves that Π_{Bmac} is secure. $\square$

3 Hash functions

3.1 Constructing New Hash Functions

Hash functions are generally built from compression functions that are able to compress a given input into an output of smaller length, whilst still preserving the property of collision resistance.

We will now consider how to develop hash functions with large compressions from hashes with small compressions. The least compression function on a hash is a 1-bit compression function. This function takes an input of $\{0, 1\}^{n+1}$, and outputs $\{0, 1\}^n$, thereby reducing the length by one bit.

We now argue that a 1-bit compression function can be used to create an arbitrarily long compression function. We will show this by first creating a 2-bit compression function from the 1-bit compression function.

Say we have a hash H with a 1-bit compression; $H : \{0, 1\}^{n+1} \rightarrow \{0, 1\}^n$. Then, we can construct a new hash function H' such that $H' : \{0, 1\}^{n+2} \rightarrow \{0, 1\}^n$ through the following procedure: We split the $(n+2)$ -bit input m into two parts, m' and b . The first $(n+1)$ bits are denoted as m' and the very last bit is denoted as b . We can apply the original hash function H to m' , and because H is a 1-bit compression hash, $H(m')$ will yield an n -bit function. Since b has a length of one bit, the concatenation $H(m')||b$ is an $(n+1)$ bit function. Then, we reapply the hash H to the concatenation; $H'(m) = H(H(m')||b)$. This yields a final n -bit output. Hence, we've produced H' that takes an $(n+2)$ -bit output and yields an n -bit input. We can repeat this procedure arbitrarily many times to create hash functions with arbitrarily large compressions.

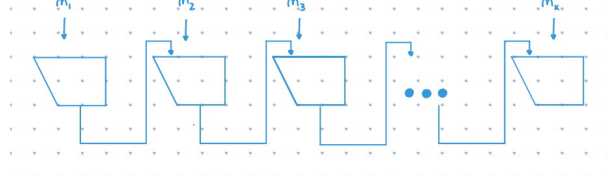


Figure 1: Merkle-Damgard Hash Chain

4 Merkle-Damgard

4.1 Merkle-Damgard Iterated Hash

See figure 1 for a visual representation of a Merkle-Damgard (MD) Hash Chain.

The concept of the overall MD Hash Chain is giving the chain a large input and it returns a small output.

For instance, we have a message m such that the whole message length is a multiple of block length b . We do this by adding zero-padding to the message to make it a multiple of b .

For each separated block, the input (visualized as m_1 in figure 1) is $h+b$ and the output is h .

Claim: We can use a MD Iterated Hash to hash any arbitrarily long message by breaking the message into bits of length b and then applying the hash on the first $h+b$ block, then the next $h+b$ block and repeating this process until the end of the message is reached. We can infinitely expand the compression function as long as we can prove that H is secure.

4.1.1 Proving that H is secure

Given $H : \{0, 1\}^{n+1} \rightarrow \{0, 1\}^n \rightarrow$ CRHF Construction $H' : \{0, 1\}^{n+2} \rightarrow \{0, 1\}^n$

$$H'(m) = H(H(m_{1\dots n+1} || m_{n+2}))$$

where $m_{1\dots n+1}$ is the first $n+1$ bit and we are concatenating that to the $n+2$ th bit of m .

Prove H is secure

Proof.

Claim: if H is a CRHF then H' is also CRHF

An attacker finds m_1 and m_2 of size $n+1$ and $n+2$ respectively, such that $H'(m_1) = H'(m_2)$.

First, compute $H(m_1)$, then we find the hash.

There are only 2 possibilities, m_1 and m_2 are either different or equal. If they are equal, a collision will be found when the hashes of both m_1 and m_2 are taken.

If $m_1 \neq m_2$, then we found two messages such that $H(m_1) = H(m_2)$.

If the smallest bit is collision resistant, then the bigger bits are also collision resistant. m_1 applies the Hash function. If the first one is secure then the whole chain is secure

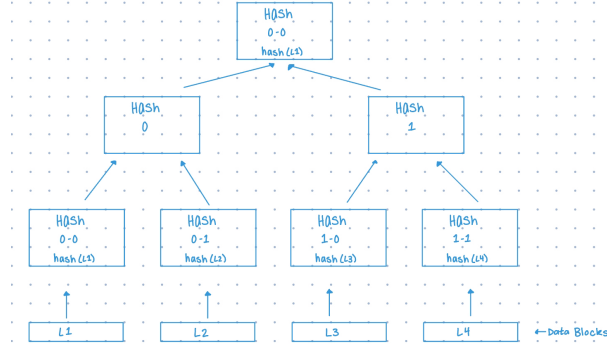


Figure 2: Merkle Tree

5 Merkle Trees

5.1 Merkle Tree Hashing

Definition 5.1 (Merkle Tree Hashing). A Merkle Tree (shown in figure 2) can take in a large message and hash it down to smaller bits in order to use a single hash function that handles 1 bit compression and expands that after using a MD Hash Chain or a Merkle Tree.

At the root of the tree is the hash of the message. As a real world example, this is used in things such as bitcoin since the processes can run in parallel to the entire message rather than relying on the previous processes to complete, and without requiring the exact amount of blocks as the chain.

Weaker properties of Hash Functions

Definition 5.2 (Second Pre-Image resistance). Given x , it is hard to find x' , such that $H(x) = H(x')$ and $x \neq x'$.

In other words this is stating that it is hard to find two outputs of the Hash function such that the inputs are not the same.

Definition 5.3 (Pre-image resistance). Given $y \in \{0,1\}^h$, it is hard to find $x \in \{0,1\}^*$ such that $H(x) = y$.

In other words this is stating that given the output y from a hash H , it is hard to find the input x .

Property: Collision Resistance $\Rightarrow$ 2nd pre-image resistance $\Rightarrow$ Pre-image resistance

This reads in English as "If you can win the pre-image challenge, you can win 2nd pre-image challenge, and if you can win the 2nd pre-image, then you can find a collision." In collision resistance the attacker is given nothing. This is why it is essentially the hardest to solve and each after is easier since the attacker is given more and more useful information.

6 Birthday Paradox

Definition 6.1 (Birthday Paradox). The Birthday Paradox is as follows: "If there is a room with 23 people, there is a 50% chance that exactly 2 people have the same birthday"

The 23 people is derived from the $\sqrt{365}$ of 365 possible birthdays times a small constant.

Definition 6.2 (Pigeonhole Principle). The pigeonhole principle states that if there are $n+1$ items placed into n items at least one bucket must contain more than one item.

The pigeonhole principle is an example of the worst case scenario. In relation to the birthday paradox, we can view that defenders need to analyze the worst case to account for all possibilities. But the attacker can avoid this by looking for the average case since they only need to be right 1 time to win whereas defenders need to be right all the time to win.

6.1 Brute forcing the birthday paradox

$$H(1), H(2), \dots, H(i)$$

writing out the probability that each person has a unique birthday by stating the probability that each of their birthdays is different than the birthdays before it

$$1 \times \frac{364}{365} \times \frac{363}{365} \dots \frac{365-n}{365} = e^{-\frac{n(n-1)}{2 \times 365}} = e^{-\frac{n^2}{2 \times 365}} \rightarrow \frac{1}{2}$$

As long as

$$n = \sqrt{365}$$

There is a 50% chance that 2 people have the same birthday
The birthday paradox was continued in the next lecture.